

Adaptive Cruise Controller for RC Car Using STM32

Erickson A. Mijangos

Department of Electrical Engineering
California State University Fullerton, Fullerton, California
eamijangos@csu.fullerton.edu

Dr. Mike Turi

Department of Electrical Engineering
California State University Fullerton, Fullerton, California
mturi@fullerton.edu

Abstract—This project documents the design and development of an RC Car built from the ground up, as an attempt to implement and test an Adaptive Cruise Control System. Two STM32 Nucleo-Boards from STMicroelectronics are used: the STM32L432KC for the transmitter and the STM32H723ZG for the RC Car. A Python User Interface is developed for controlling the RC Car and performing data acquisition in order to analyze and develop the adaptive cruise controller.

The sensors integrated to the RC Car are the LIS3DH accelerometer and the Garmin LIDAR Lite V3. The accelerometer was used to determine the velocity of the RC Car, and the LIDAR to measure the relative distance to a leading vehicle ahead. During testing, two challenges arose: the velocity calculated by the accelerometer drifted from the true value over time due to sensor noise, and the NRF24L01 lost communication for 1–1.5s every 20ms cycle which affected velocity calculation.

Due to Earth's gravitational influence on the accelerometer, a calibration feature was implemented, but the communication losses made it difficult to confirm its effectiveness. Based on the test results, the adaptive cruise controller could not be fully implemented. Future work includes integrating a Hall Effect sensor and applying a Kalman Filter for sensor fusion to improve velocity accuracy and enable closed-loop speed control.

Index Terms—Adaptive Cruise Control, STM32, PID Controller, LIDAR, NRF24L01, PyQt5, UART.

I. INTRODUCTION

The purpose of this project is to explore electronics, software development, sensor integration, numerical analysis, real-time data acquisition, and primarily control engineering by building an RC Car from the ground up. The RC Car is designed to develop an Adaptive Cruise Controller with data acquisition in order to properly implement it. Figure 1 shows a high-level overview of the entire project — the RC Car listens and sends data back to the transmitter, which is connected to a Python User Interface. The RC Car and transmitter communicate using the NRF24L01 programmable RF module, and the transmitter connects to the computer via USB using the UART protocol to send commands and receive data.

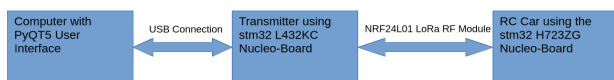


Fig. 1. High-level overview of the project architecture.

The setup of the Transmitter and Python User Interface is shown in Figure 3, and the RC Car with the transmitter is shown in Figure 4. The RC Car integrates the LIS3DH accelerometer to measure velocity and the Garmin LIDAR Lite V3 to measure relative distance.

Based on a simulation study conducted in MATLAB/Simulink at CSULB¹, the Adaptive Cruise Control system is designed to autonomously maintain the desired velocity while detecting and measuring the relative distance to a leading vehicle ahead. As shown in Figure 2, the relative distance and vehicle speed are analyzed to tune the adaptive cruise controller.

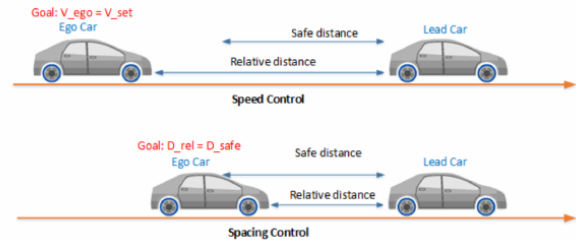


Fig. 2. Speed/Spacing Control to setup Adaptive Cruise Control.

To implement the adaptive cruise controller in hardware, the Garmin LIDAR Lite V3 serves as the primary distance sensor, measuring the relative distance between the Ego Car (the RC Car under ACC control) and the Leading Car (a second RC Car given a specified motion). The LIS3DH accelerometer measures the Ego Car velocity, in which a PID controller can be developed to make the RC Car autonomous. The Leading Car will be given a speed with a small oscillation to simulate realistic driver behavior, as shown in Figure 5.

To achieve the simulated behavior of the adaptive cruise controller, the control system shown in Figure 6 models the Leading Vehicle motion and the Sensor Model, which depends on the relative position and velocity of the leading vehicle and the ego vehicle, so it can be fed into the ego vehicle plant to predetermine next motion. The input signals measured from the relative and ego vehicle parameters are used to determine the ego vehicle Adaptive Cruise Controller are shown in Figure 7.

¹Full report available in the project repository: EE471_ACC_CSULB.pdf.

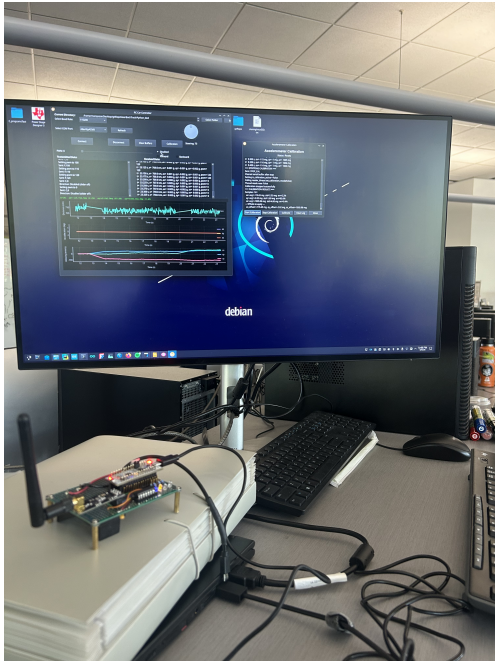


Fig. 3. The Transmitter connected to the Python user interface.

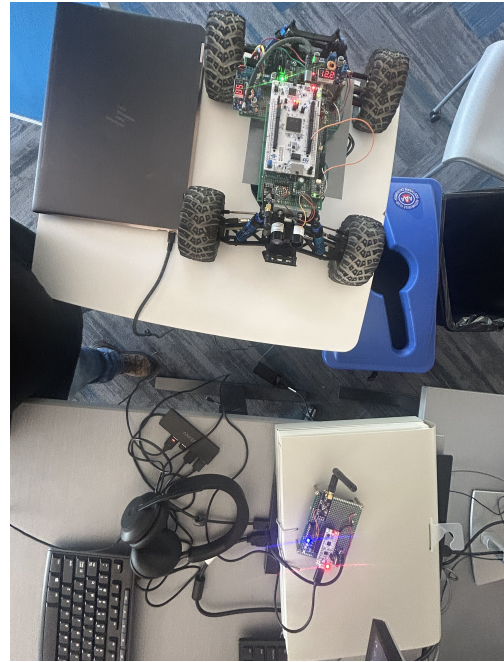


Fig. 4. The Transmitter and RC Car.

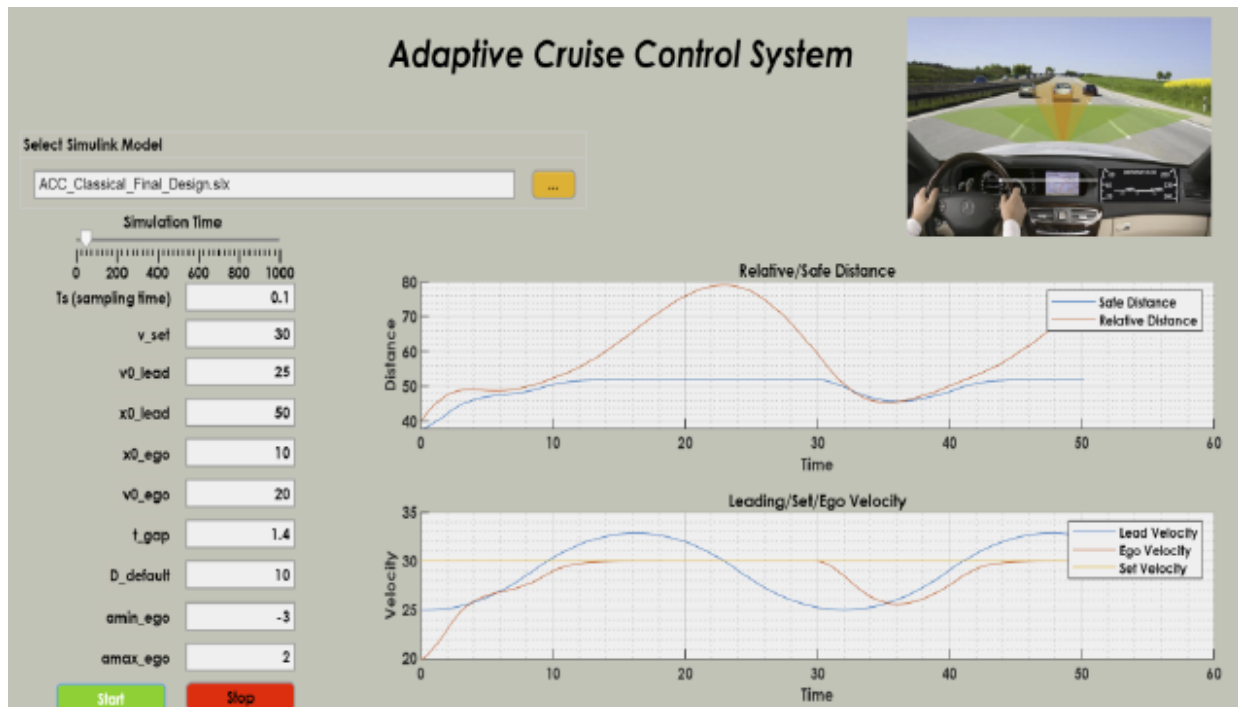


Fig. 5. The User Interface developed in MATLAB and Simulink.

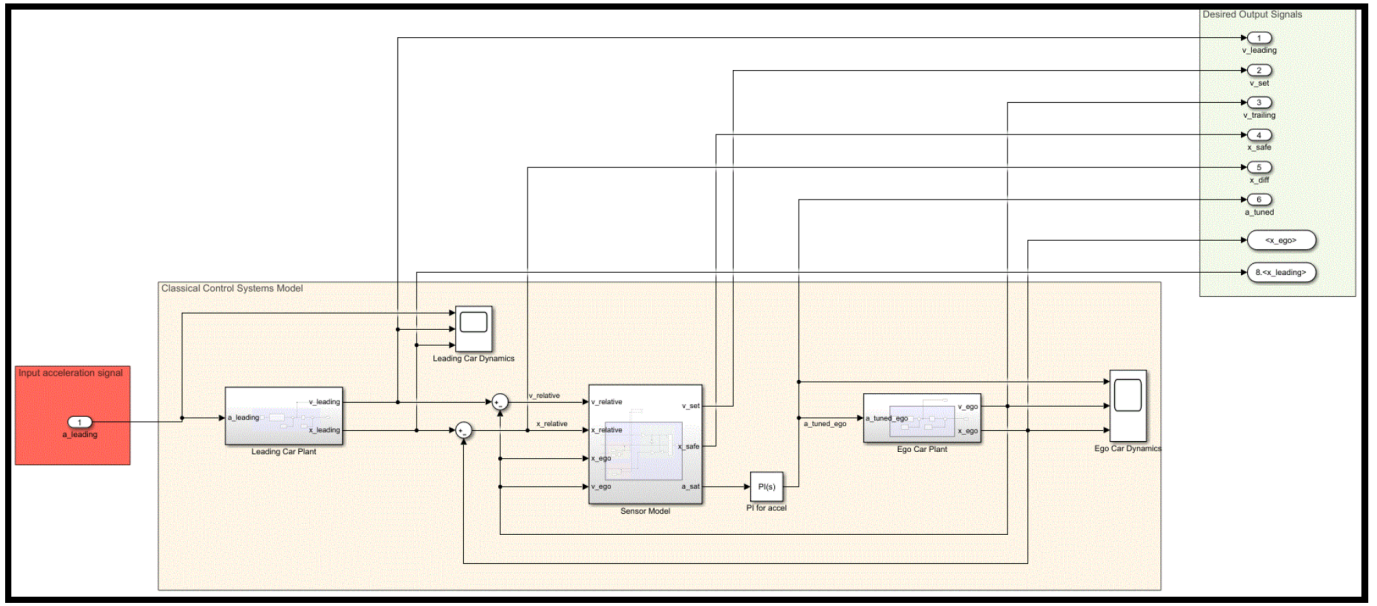


Fig. 6. High-level overview of ACC model.

To place the adaptive cruise controller into test, data acquisition with a decent sample rate from both accelerometer and LIDAR are required. To analyze and process the signals collected, the python user interface needs to be implemented in order to acquire data in real time so a proper adaptive cruise controller can be implemented.

II. HARDWARE ARCHITECTURE

A. RC Car Design

The RC Car uses the STM32H723ZG Nucleo-Board, due to having the 550MHz clock speed, to achieve better timer control if necessary, minimize the execution time per instruction/code, accessibility of more peripherals, and the larger system RAM to store large amount of data for testing purposes. The peripherals used in this MCU are SPI, USART, I2C, timers, and Direct Memory Access to either control sensor and transmit data during development. In Figure 9, shows the STM32H723ZG Nucleo-Board integrated with all the sensors, and power system section along with the RC Car with the all sensors/modules labeled in Figure 8.

The Entire RC Car is powered with a 12V battery, but the RC Car is divided by subsections based on the schematic. On the left there a yellow box is drawn to show the MCU along with its sensors, and on the right in the green box shows the power system and the motor driver. The power section is electrically isolated due to the FOD8001 Logic Optocouplers, and the TMH1205S 2W Traco Isolated DC-DC Converter that isolate the power ground and digital ground on the MCU section. This feature is added due to capability to troubleshoot, isolate noise that can influence the sensors during data transmission, and if any power section components/modules are destroyed it would not destroy the MCU and the sensors.

As labeled in the RC Car schematic the sensors used are labeled to their corresponding pins on the MCU and their protocols. The NRF24L01 RF module uses the SPI protocol where the data rate is set to 3.9 Mbit/s. According to the datasheet of the NRF24L01 would need 130us to switch RX or TX mode, and also to including the execution timing of the program which can add more time. When bidirectional communication was first tested, the bare minimum to have

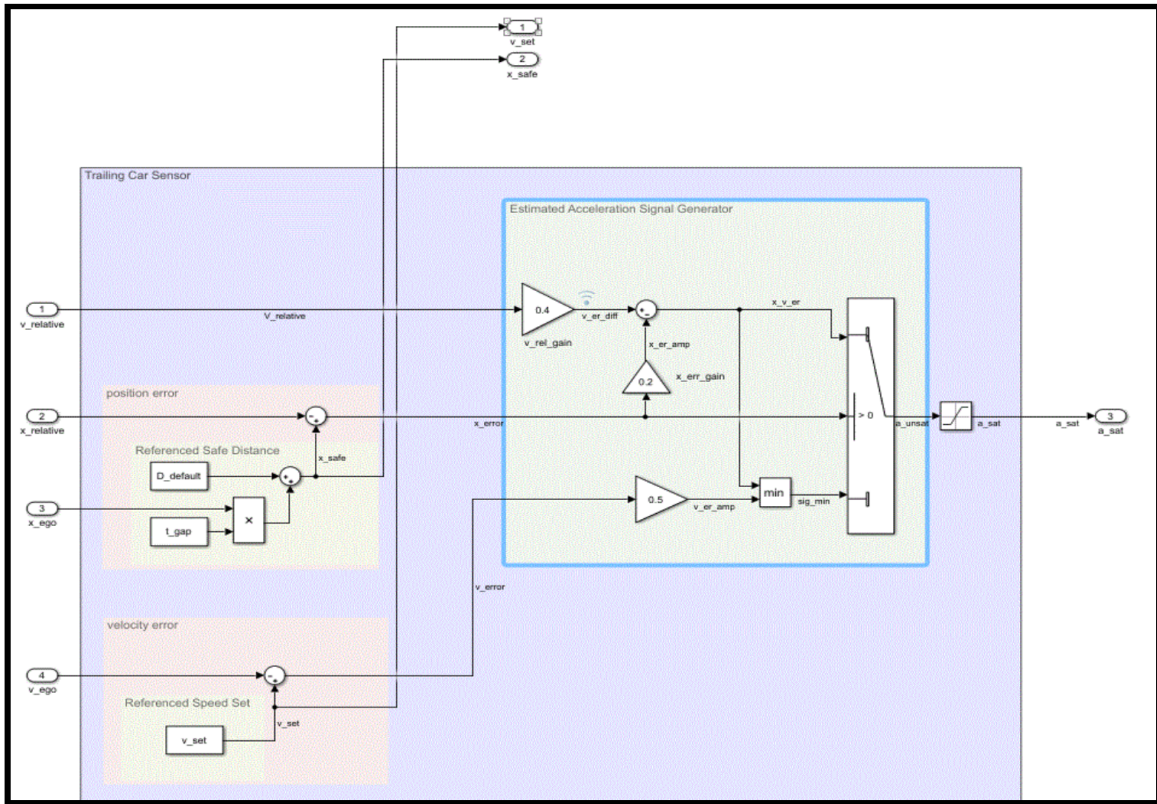


Fig. 7. Block diagram of vehicle plant and derivation for velocity and position motion.

a reliable communication is 4ms. To provide reliable bi-directional communication for most of the time is 10ms before toggling RX/TX mode. The 10ms toggling time also considers the amount of time to read the sensor data and package the data for transmission. On the MCU, three built-in LEDs are used to track the state of the RC Car firmware. Adding the status LEDs helps the user keep track and troubleshoot the RC Car during the development or issue.

- **Yellow LED** — Transmitter is in TX mode (sending sensor data to transmitter)
- **Red LED** — Data packet transmitted from RC Car
- **Green LED** — Command received from transmitter for RC Car next action

The remaining two sensors that uses I2C protocol are the LIS3DH accelerometer, and Garmin LIDAR V3 Lite. The clock speed that the MCU can configure for I2C protocol is either 100kHz or 400kHz. Both sensors are clocked to 400kHz, and was confirmed that it is reliable when both sensors library were written. The perf board that holds the MCU, have the wires soldered and close to the MCU in order to ensure I2C is reliable and minimize the stray capacitance that can affect the rise and fall times of both data and clock lines. Both sensors would be sampled when it is in the state to transmit data to the transmitter which is every 20ms.

With the MCU and the sensors isolated from the power section, the FOD8001 optocouplers and servo motor needs to still be powered. The 12V serving as the main power source

of the RC Car, the MCU, optocouplers, and motor driver still need to be powered properly by adding voltage regulators, and DC/DC converters. The optocoupler on the power section, still needs to be powered about 5V so that the PWM pulse can be passed to the motor driver and servo motor.

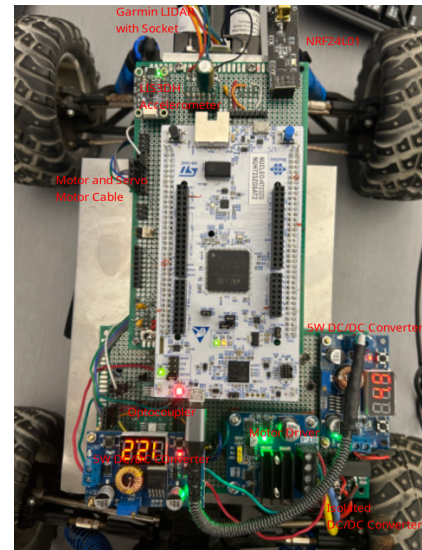


Fig. 8. RC Car Labeled.

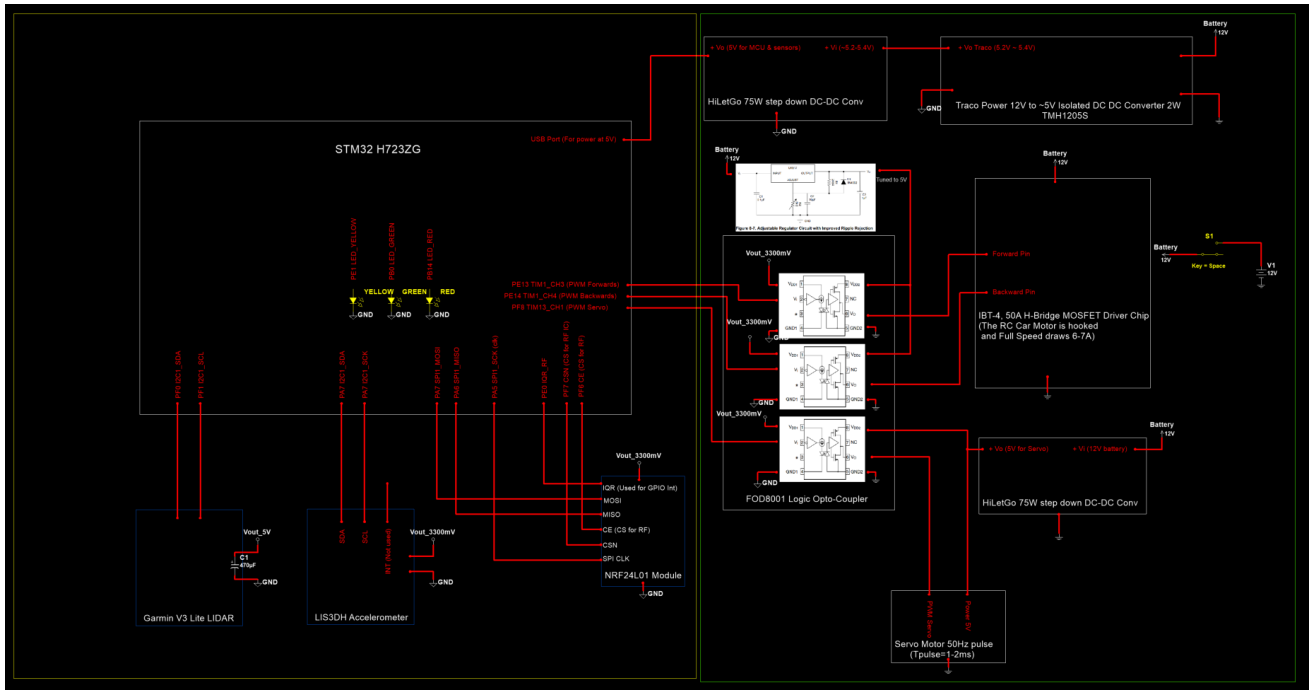


Fig. 9. The Schematic of RC Car.

Two of the optocouplers responsible to pass the PWM pulses to the MOSFET motor driver, are powered by the LM317 voltage regulator, where it can be tuned 5V based on the adjustable circuit from the Texas Instruments datasheet as shown in the schematic. For both the third optocoupler and servo motor, are powered by the 75W HiLetgo DC/DC stepdown converter. Although the both the third optocoupler and servo motor can be powered the LM317, it was not due to potential concerns that the it would heat up and go to thermal runaway. The servo draws about 500-700mA when it was manually tested with 5V supply along with function generator, generating the PWM that is pulsing within 1-2ms, with a time period of 20ms (PWM frequency is 50Hz). Since the LM317 drops the 12V to 5V, it would have a 7V drop across it and if more current is drawn it would dissipate more heat. As a result it is a safer route to use the 75W HiLetgo DC/DC stepdown converter to power the optocoupler along with the servo motor.

The 75W HiLetgo DC/DC stepdown converter can display the input and output voltage it was provided. The battery can be monitored from checking the HiLetgo DC/DC converter that is intacted to the servo motor and third optocoupler but toggling the output push button. The traco isolated DC/DC converter is stepdown to about 5.2V-5.4V from the 12V battery, but a HiLetgo DC/DC converter is added between the output of the isolated DC/DC converter and the MCU so that it can be tuned to 5V. It can be powered by the isolated DC/DC converter but according to the schematic of the H723ZG Nucleo-Board on the Micro-USB port it has a zener diode to clamp the supply voltage of the entire board to 5V and would be safer to tune it at 5V. When the battery

runs low, the bare miminum voltage that be supplied to the MCU to work proper is 4.6V before it powers off and become unreliable.

B. Transmitter Design

Compare to the RC Car, the schematic of the transmitter is simpler as shown in Figure 11, along with the actual labeled transmitter in Figure 10. The Nucleo-L432KC board was chosen for the transmitter due to it small Arduino Nano like size, and the max clock speed is 80MHz. The performance does not need to be robust compared to Nucleo-H723ZG board since primarily it would be toggling every 10ms from RX/TX mode and USART interrupts are used to send commands to transmitter from the Python User interface and transmit data it has receive to the User Interface. The transmitter is connected to the computer to transmit/receive data by using the UART protocol with a baud rate of 115200 bits/s.

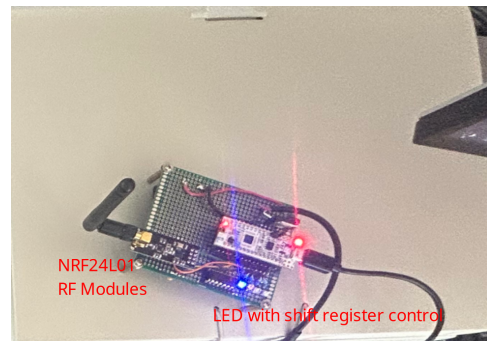


Fig. 10. Transmitter Labeled.

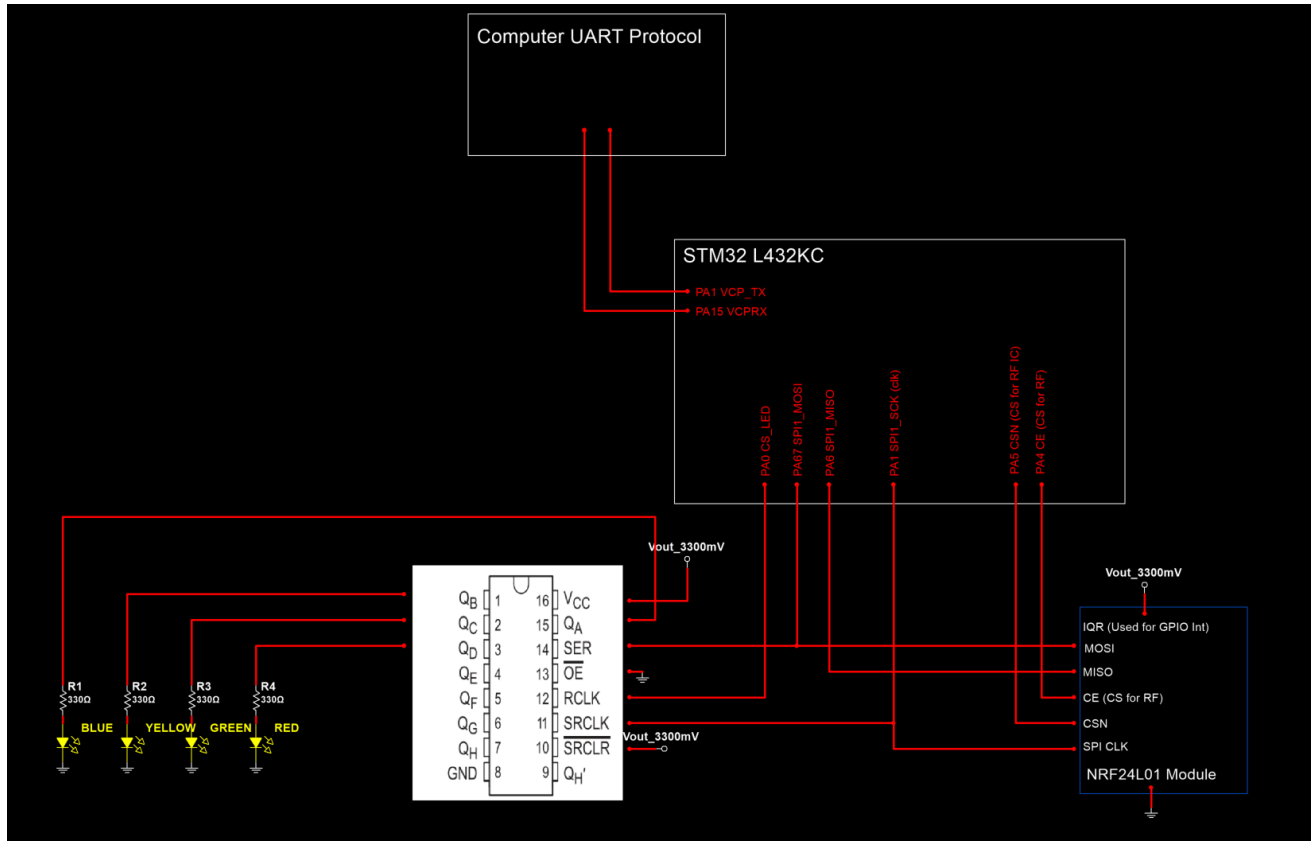


Fig. 11. The Schematic of Transmitter.

Along with the NRF24L01 RF module, a shift register is added to control four status LEDs where:

- **Blue LED** — Transmitter is in RX mode (listening for RC Car data)
- **Yellow LED** — Transmitter is in TX mode (sending command to RC Car)
- **Red LED** — Data packet received from RC Car
- **Green LED** — Command received from Python GUI and ready to transmit

Adding the status LEDs help the user track the actions of the transmitter for trouble-shooting and development.

III. FIRMWARE DESIGN

A. RC Car Firmware Design

Based on the hardware integration of the RC Car, the firmware designed in Finite State Machine (mostly behave as Mealy State Machine where next state varies on both current states and inputs to determine the state). The heart of the firmware is Timer 3, where it will toggle between RX/TX mode and will determine the next action based on the current inputs it has receive during any current state it is in. Figure 12 shows the RC Car Finite State Machine logic where in the center most of the action occurs the Timer 3 periodic interrupt to toggle communication. To simplify the logic further pseudocode is added to conditionally determine the RC Car next action.

The movement of the RC Car including the forward, backward, coast, brake, and steer states will vary on the next state variable which is determine what command it has received in the Receive state. If the RC Car is in disable mode, it will keep the RC Car in coast mode next state variable will be marked as coast state and it will not transmit data back to the transmitter. In the Transmit state there is a stopStateFlag where that will keep the RC Car disabled, if next state is marked as wait state, then it will signal the RC Car stay stationary and not transmit sensor data.

When the RC Car receives either forward, backward, or steering commands it will start to transmit sensor data. In the receive state for the entire 10ms, it will be in a poll state until it has receive a command from the NRF24L01 RF module. If a command is receive, it will mark the current state to get sensor data and the next state after it has acquired sensor data to the RC Car Action which either can forward, backwards, or steering.

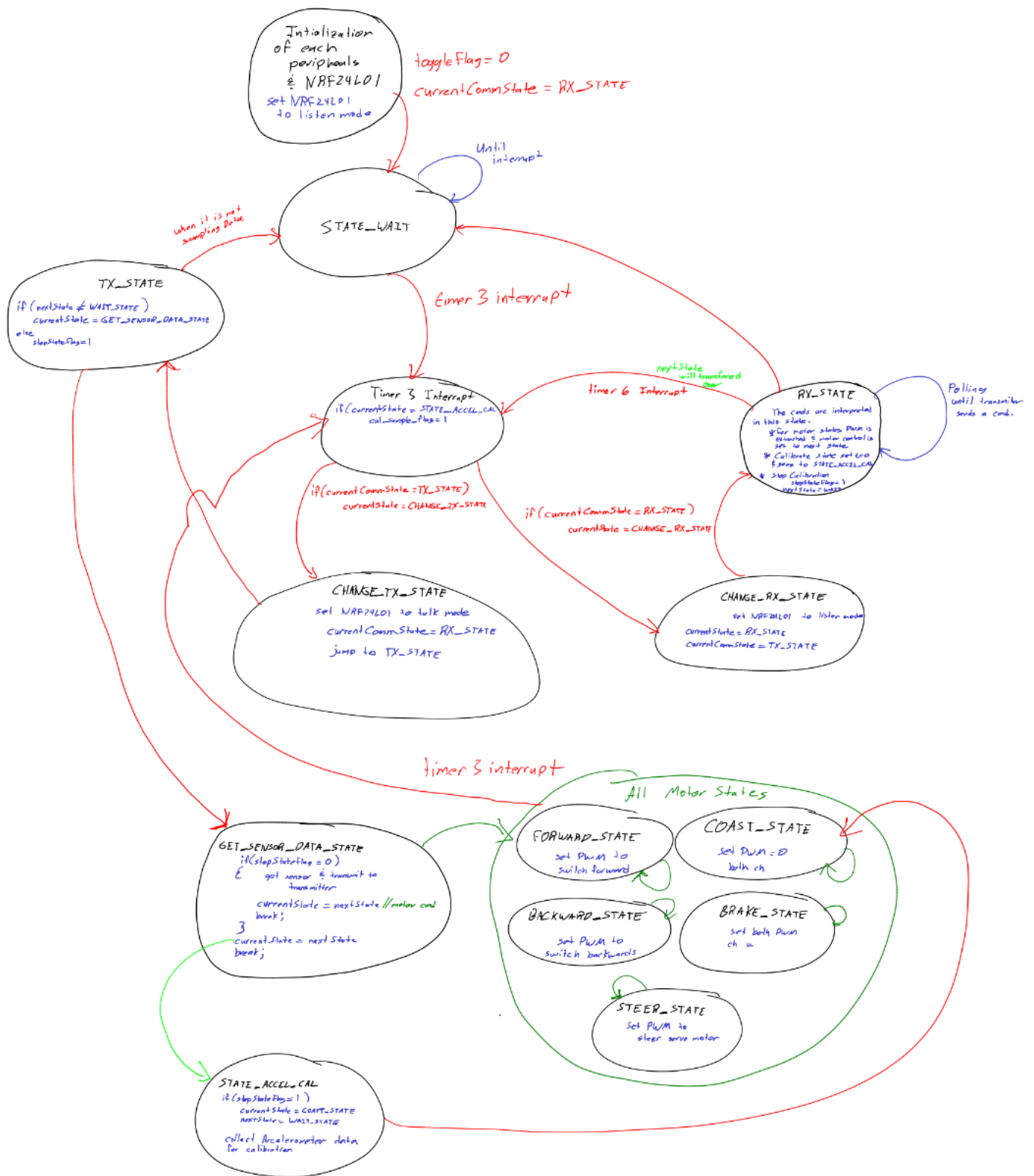


Fig. 12. The finite state machine of the RC Car.

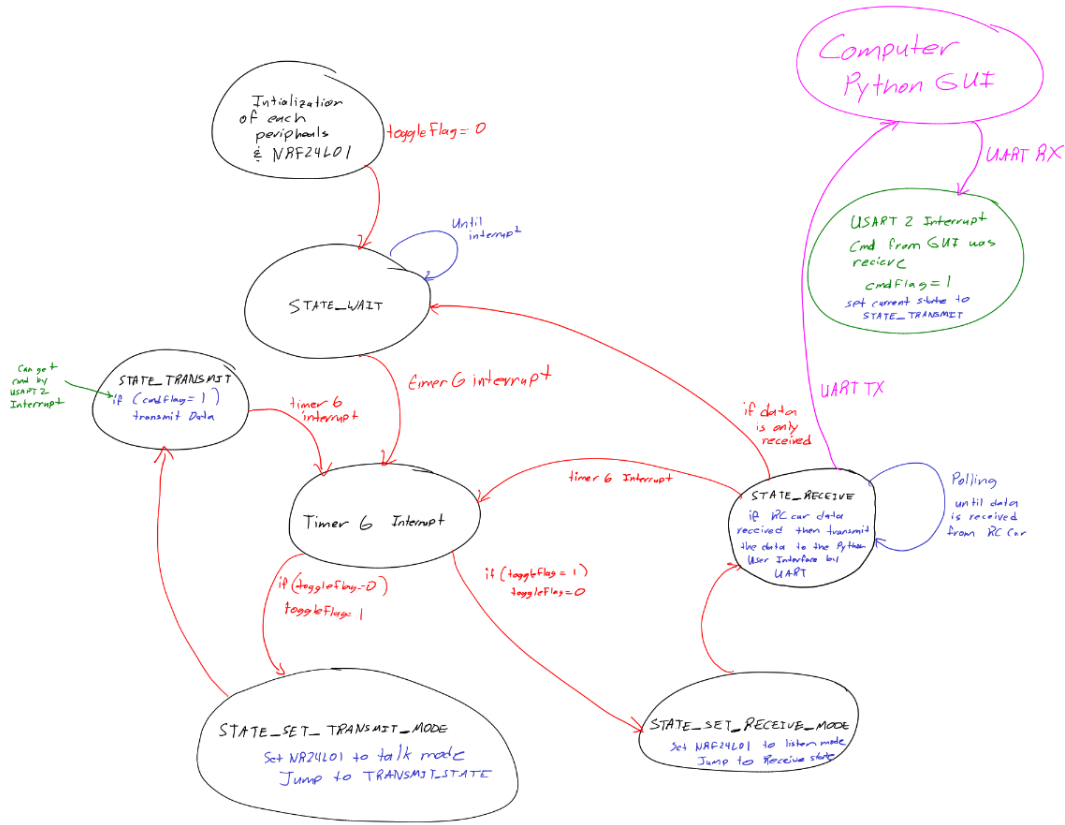


Fig. 13. The finite state machine of the Transmitter.

In the get sensor data state, it checks if stopStateFlag where if set to zero, it can start getting the data from the sensors. It was added due to the calibration feature that was added, for calibrating the accelerometer. The difference is that the get state will package the data in a 32-byte format in the following notation:

```
timestamp_ms,distance_cm,ax_mg,ay_mg,az_mg\n
```

Whereas the calibration format is excluding the LIDAR measurements in the following notation:

```
timestamp_ms,ax_mg,ay_mg,az_mg\n
```

The calibration state was added since the accelerometer is measuring Earth gravity, and since change of motion is desired the accelerometer is transmitted to calculate the average offset.

B. Transmitter Firmware Design

The transmitter finite operates in similar fashion (Figure 13) with the difference of only listening for data, and transmits data when a USART interrupt occurs with a baud rate of 115200 bits/s. The USART interrupt, signals the MCU that it has received a command from the Python User interface will prepare to transmit command when it switches to the transmit state. When data gets received during the receive state, it will transmit the data it has received to the Python User interface.

IV. PYTHON GRAPHICAL USER INTERFACE

The transmitter will transmit commands to the RC Car, and receive data from the RC Car under the Python User interface. Figure 14. The package used for the user interface is the PyQt5, PyQtgraphs for the real time plots, and PySerial to perform UART communication between the user interface and the transmitter. The actions that were performed are displayed in two textboxes labeled Transmitted Box Data which shows which buttons are pushed and sent to the transmitter and the Receive Data Box to display the data that was received from the transmitter. Similarly the calibration window has a textbox to display both actions and data received based on the buttons pushed.

In order to display the textbox and plot data in real time, the user interface would need to use the QThread which imported under the PyQt5 package, otherwise the user interface would not respond. The current flowchart/state diagram does not include the logic of determining whether the data is measurement (all sensors measured) or accelerometer data (for calibration) but the functionality stays the same as shown in Figure 15. As mentioned in the firmware section of the RC Car the disable button ensures the RC stays stationary and will collect data when the steering dial is moved, or both forward/backward radio are set with the adjusted slider that will send the PWM value to control speed.

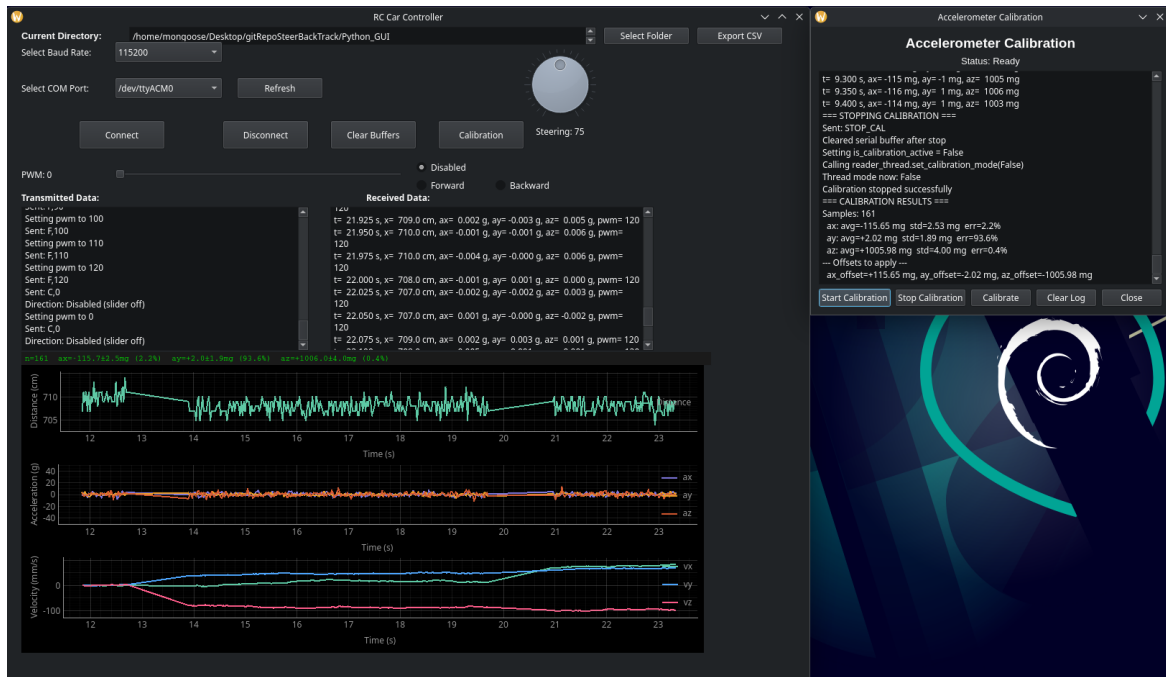


Fig. 14. The Python including Calibration window.



Fig. 15. The state diagram for threading feature when data is received from transmitter.

V. EXPERIMENTAL RESULTS

Your results go here.

VI. CONCLUSION

Your conclusion goes here.

REFERENCES

- [1] STMicroelectronics, *STM32H723ZG Datasheet*, Rev. 4, 2022.
- [2] Garmin Ltd., *LIDAR-Lite v3 Operation Manual*, 2017.